

Alloy: A Quick Reference and an interpretation into B

Marc Frappier, 2020-11-09
version 2.1

Inspired from the document [Alloy Quick Reference](https://www.monperrus.net/martin/alloy-quick-ref.pdf) written by [Martin Monperrus](https://www.monperrus.net/martin/alloy-quick-ref.pdf)
<https://www.monperrus.net/martin/alloy-quick-ref.pdf>

Alloy Specification

The typical structure of an Alloy specification is as follows

- Declaration of signatures
- Declaration of facts
- Declaration of predicates and functions
- Run statement and check statements

However, these can be freely mixed (ie, no ordering is imposed on the declarations).

Alloy Expressions

- Basic types are declared using signatures.
- A signature declares a set of atoms.
- An expression is either a term or a formula.
- A type can be a signature or a term constructed using signatures.
- A variable v must be typed using the declaration $v : T$, where T is a term constructed using signatures.
- Variable names can be primed ; ex: x , x' , x'' , x'''
- Alloy terms (ie, values other formulas) are *nary*-relations.
 - Alloy has no explicit notion of sets, tuples or scalars; a term is a *nary*-relation
 - A tuple is represented using a singleton relation.
 - A scalar is represented using a singleton, unary relation
 - A set is represented using a unary relation.

Alloy terminology (as defined in Daniel Jackson's book *Software Abstractions : Logic, Language, and Analysis*)

- A **model** is an Alloy specification
- A **fact** is a formula that must be satisfied by a model instance
- A **model instance** is an assignment of values to the symbols (signature and relations) that satisfies the facts and the signature constraints of a specification.

- This is a bit confusing wrt to the usual terminology in logic: a model in logic is what is called a model instance in Alloy.
- A **signature** is a set of atoms of the same type; a signature also denotes a type whose value is its set of atoms.
- A **field** is declared in a signature and it denotes a relation. A field may have constraints on its values (**one**, **lone**, **set**).
- An **atom** is an element of a signature. An atom is a unary relation with only one element (ie, a singleton set).

Signatures

<i>Notation</i>	<i>Intuitive Meaning</i>	<i>Equivalent B declaration</i>
<code>sig Book {...}</code>	Declares a set <code>Book</code>	SETS <code>Book</code>
<code>sig Book { author: Author }</code> <code>sig Author {...}</code>	Declares a set <code>Book</code> , and a total function <code>author</code>	SETS <code>Book</code> , <code>Author</code> CONSTANTS <code>author</code> PROPERTIES <code>author : Book --> Author</code>
<code>sig Book { author: set Author }</code>	Declares a set <code>Book</code> , and a relation <code>author</code> which is a subset of the Cartesian product $\text{Book} \times \text{Author}$	... PROPERTIES <code>author : Book <-> Author</code>
<code>sig Book { author: some Author }</code>	a book has at least one author	PROPERTIES <code>author : Book <-> Author</code> <code>dom(author) = Book</code>
<code>sig A { f: lone B }</code>	<code>f</code> is a partial function from <code>A</code> to <code>B</code>	<code>f : A +-> B</code>
<code>sig A { f: B }</code>	<code>f</code> is a total function from <code>A</code> to <code>B</code>	<code>f : A --> B</code>
<code>sig A { f: one B }</code>	<code>f</code> is a total function from <code>A</code> to <code>B</code>	<code>f : A --> B</code>
<code>sig A { f: set B }</code>	<code>f</code> is a relation from <code>A</code> to <code>B</code>	<code>f : A <-> B</code>
<code>sig Dictionary extends Book {...}</code> <code>sig Novel extends Book {...}</code>	Inheritance, all extension signatures are disjoint.	CONSTANTS <code>Novel</code> , <code>Dictionary</code> PROPERTIES <code>Dictionary</code> $\subseteq$ <code>Book</code> & <code>Novel</code> $\subseteq$ <code>Book</code> & <code>Novel</code> $\cap$ <code>Dictionary</code> = <code>{}</code>
<code>abstract sig Book {...}</code> <code>sig Dictionary extends Book {...}</code> <code>sig Novel extends Book {...}</code>	Abstract signature, has no proper instance; all instances are obtained from extensions	PROPERTIES ... <code>Novel</code> $\cup$ <code>Dictionary</code> = <code>Book</code>
<code>one sig Bible extends Book {...}</code>	Singleton, $ \text{Bible} = 1$, <code>Bible</code> subset of <code>Book</code>	PROPERTIES <code>Bible</code> $\subseteq$ <code>Book</code> & <code>card(Bible)</code> = 1
<code>sig LNCS in Book {...}</code>	<code>LNCS</code> subset of <code>Book</code> . It may overlap with other extensions of <code>Book</code>	PROPERTIES <code>LNCS</code> $\subseteq$ <code>Book</code>

Boolean Operators

p and q, p && q	Conjunction	p & q
p or q, p q	Disjunction	p or q
p implies q, p => q	Implication	p => q
p implies e1 else e2	Conditional expression (e1, e2 can be of any type or a formula)	B allows implication only between formulas (p => e1) & ((not p) => e2)
p iff q, p <=> q	Equivalence	p <=> q
not p, !p	Negation	not p

Quantification

all x1,...,xn : S1, ..., y1,...,yn : Sm p	Universal quantification	!(x1,...,xn,..., y1,...,yn). (x1 : S1 & ... & xn : S1 ... & ... & ... y1 : S2 & ... & yn : Sm => p)
some x1,...,xn : S1, ..., y1,...,yn : S2 p	Existential quantification, at least one	#(x1,...,xn,..., y1,...,yn). (x1 : S1 & ... & xn : S1 ... & ... & ... y1 : S2 & ... & yn : Sm & p)
one x : S p	Exactly one assignment of values to variables satisfies p. Also allowed for list of variables.	#(x).(x : S & p) & !(x1,x2). (x1:S & x2:S & p[x:=x1] & p[x:=x2] => x1=x2)

$\text{no } x : S \mid p$	No assignment of values to variables satisfies p . Also allowed for list of variables.	$\text{not } (\#(x).(x : S \ \& \ p))$
$\text{lone } x : S \mid p$	At most one assignment of values to variables satisfies p . Also allowed for list of variables.	$(\dots \text{one } \dots) \text{ or } (\dots \text{no } \dots)$

Sets (ie, unary relations)

none	The empty set	$\{\}$
univ	All instances of all types (the universe)	N/A
Int	set of integers, defined in module <code>util/integer</code> The range of integers is defined by the scope <code>run ... for n int</code> where n is the number of bits used to represent a signed integer. Thus, the range is $-2^{n-1} \dots (2^{n-1})-1$. ex: <code>for 3 int</code> is the interval $-4 \dots 3$	NAT with $\text{MININT} = -2^{n-1}$ and $\text{MAXINT} = (2^{n-1})-1$

Predefined Binary relations

iden	Identity relation on <code>univ</code> , ie, the relation $\{x:\text{univ}, y:\text{univ} \mid x=y\}$	not available The B expression <code>id(S)</code> is the Alloy expression <code>S <: iden</code> where <code><:</code> is Alloy's prerestriction operator
------	--	--

Predicates on relation

$\text{no } x$	Empty set	$x = \{\}$
$\text{some } x$	Relation not empty	$x \neq \{\}$
$\text{one } x$	$ x = 1$	$\text{card}(x) = 1$
$\text{lone } x$	$ x \leq 1$	$\text{card}(x) \leq 1$
$a \text{ in } B$	Subset or equal	$a <: B$
$a = b$	Equality	$a = b$
$a \neq b$	Inequality	$a \neq b$

Operators on relations

$a \rightarrow b$	Cartesian product $a \times b$	$a * b$
$\{x1:S, \dots, xn:Sn \mid p\}$	Set of tuples	$\{(x1, \dots, xn) \mid x1:S1 \ \& \ \dots \ \& \ xn:Sn \ \& \ p\}$ type of set elements is $((S1*S2)^* \dots)^*Sn$
$b.author$	Field access. Same as set of images of b by relation $author$	$author[\{b\}]$
$r1.r2$	Relation product	$r1;r2$ (only when $r1$ and $r2$ are binary relations) Alloy has n-ary relations; B only has binary relations
$a.b$	Relational product extended to arbitrary <i>nary</i> -relations	N/A
$b[a]$	same as $a.b$	$b[a]$ works only if b is a binary relation and a is a set
$x + y$	Union	$x \ \vee \ y$
$x \ \& \ y$	Intersection	$x \ /\ \ y$
$x - y$	Difference	$x \ - \ y$
$a <: b$	Domain restriction of relation b by set a	$a < b$
$b :> a$	Range restriction of relation b by set a	$b > a$
$\sim a$	Inverse	$a \sim$
$*a$	Reflexive-transitive closure	$closure(a)$
a	Transitive closure	$closure1(a)$
$a++b$	Relational override, ie, returns $((univ - (b.univ)) <: a) + b$	$a <+ b$
$\#a$	Cardinality	$card(a)$

B Operators not available in Alloy, with equivalent expressions in Alloy

$r.univ$	domain of r	$dom(r)$
$univ.r$	range of r	$ran(r)$

Types, constraints and multiplicities

$r \text{ in } T \rightarrow U$	Relation from T to U	$r \text{ in } T \leftrightarrow U$
$r \text{ in } T \rightarrow \text{one } U$	Total function from T to U	$r \text{ in } T \dashrightarrow U$
$r \text{ in } T \rightarrow \text{lone } U$	Partial function from T to U	$r \text{ in } T \dashrightarrow U$
$r \text{ in } T \text{ lone } \rightarrow \text{lone } U$	Partial injection from T to U	$r \text{ in } T \rightarrowtail U$
$r \text{ in } T \text{ lone } \rightarrow \text{one } U$	Total injection from T to U	$r \text{ in } T \rightarrowtail U$
$r \text{ in } T \text{ some } \rightarrow \text{lone } U$	Partial surjection from T to U	$r \text{ in } T \twoheadrightarrow U$
$r \text{ in } T \text{ some } \rightarrow \text{one } U$	Total surjection from T to U	$r \text{ in } T \twoheadrightarrow U$
$r \text{ in } T \text{ one } \rightarrow \text{lone } U$	Partial bijection from T to U	$r \text{ in } T \rightarrowtail U$
$r \text{ in } T \text{ one } \rightarrow \text{one } U$	Bijection from T to U	$r \text{ in } T \rightarrowtail U$

Integers (operators defined in module util/integer)

plus[a,b]	Sum	a+b
minus[a,b]	Difference	a-b
mul[a,b]	Product	a*b
div[a,b]	Integer division	a/b
rem[a,b]	Remainder of a divided by b	
sum[a]	Returns the sum of the integers of set a	
a < b, a = b, a > b, a <= b, a >= b	Integer comparison	a < b, a = b, a > b, a <= b, a >= b
max[a]	Maximum of set a	max(a)
min[a]	Minimum of set a	max(a)

Global Assertions

fact { f1 ... f2 }	Formulas f1,...,fn which must be satisfied by all instances of a model. Formulas f1,...,fn are implicitly conjoined.	PROPERTIES f1 & ... & fn
--------------------------------	---	-----------------------------

Syntactic Sugar

<code>author[b]</code>	<code>b.author</code>
<code>author[Book]</code>	<code>Book.author</code>
<code>p1.friend[p2]</code>	<code>friend[p1,p2]</code>
<code>let v = E F</code>	Equivalent to <code>F</code> where <code>v</code> is replaced by <code>E</code>

Ordering (operators defined in module `util/ordering`)

<code>open util/ordering[State] as states</code>	Declares a total order on <code>State</code>
<code>states/first</code>	First element
<code>states/last</code>	Last element
<code>states/next[s]</code>	Next element
<code>states/prev[s]</code>	Previous element
<code>states/nexs</code>	All next elements
<code>states/prevs</code>	All previous elements

Sequences

<code>s : seq A</code>	Sequence
<code>s.append[t]</code>	Concatenation
<code>s.first</code>	Head
<code>s.rest</code>	Tail
<code>s.elems</code>	Unordered elements

Modules

<code>open util/ordering[States] as mystates</code>	Opens module <code>ordering</code> and declares <code>mystates</code> as prefix for using it (ie, <code>mystates /function</code>)	
<code>module util/ordering[exactly elem]</code>	Declares module <code>ordering</code> with parameter <code>elem</code>	

Predicates and functions

pred wrote[a:Author,b:Book] {b.author=a}	Predicate (returns true or false)	DEFINITIONS wrote(a,b) == author[{b}] = {a}
fun books[a:Author]:set Book {author.a}	Function, returns an expression of some type, here it returns a set of books	
fun nbOfBooks[a:Author]:Int {#(author.a)}	Function, returns an integer.	

Finding an instance of a model

run {...} for n	Find instances, by default with a maximum of n instances for each signature (n is some natural number).	
run {...} for 3 Book, 4 Author	Find instances with constraints on # of instances	
run {...} for 3 but 1 Author	Find instances with constraints on # of instances, here 3 instances of all signatures except Author, for which only 1 instance is used.	
pred foo[b:Book] {...} run foo for 3 but 1 Author	Find instances satisfying predicate "foo"	

Checking an assertion of a model

assert assertion1 {good_author => good_book} check assertion1 for ...	Find counter-examples violating the assertion. Same scope specification behavior as the run command	
check nom_check {good_author => good_book} for ...	Check specified assertion. Assertion has the name nom_check	
check {good_author => good_book} for ...	Check anonymous assertion	

Precedence

(In increasing order; operators on the same line have same priority)

<i>Expressions (operands are not Booleans)</i>	<i>Logical expression (operands are Booleans)</i>
$\sim, ^, *$. [] <:, :> -> & ++ # +, - no, some, lone, one, set !, not in, =, <, >, =, =<, =>	!, not &&, and =>, implies, else <=>, iff , or let, no, some, lone, one, sum (<i>quantification</i>)

All binary operators associate to the left, with the exception of implication, which associates to the right. So, for example, $a.b.c$ is parsed as $(a.b).c$, and $p \Rightarrow q \Rightarrow r$ is parsed as $p \Rightarrow (q \Rightarrow r)$.